Seminar
Integration Testing and Mockito

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. Testing basics
2. Test doubles and integration
3. Integration Testing

What is Software Testing

Definitions

- Stakeholder view: *"An investigation conducted to provide stakeholders with information about the quality of the software product or service under test"*
- Simulation view: *"Activity in which a system or component is executed under specified conditions, the results are observed or recorded, and an evaluation is made of some aspect of the system or component"*
- Process view: *"The process consisting of all lifecycle activities, both static and dynamic, concerned with planning, preparation and evaluation of software products and related work products to determine that they satisfy specified requirements, to demonstrate that they are fit for purpose and to detect defects."*

Aspects of testing

Why you test

- I.e., what is the goal?

What to test

- Subject/System Under Test (SUT)

Who prepares tests, run tests, evaluates results, etc.

- and relationships with other stakeholders (e.g., developers, customers)

When tests are prepared, executed

- w.r.t. when the SUT is built (e.g., before, after, altogether)

How testing is done

- I.e., what techniques, tools, etc.?

The various facets of testing

"Schools" of testing for programmers

- Analytic School: testing as a rigorous task, via formal methods
  - Motto: Programs are logical artifacts, subject to math-like laws
  - ⇒ A test suite is a formal specification of a systems's behaviour
- Factory School: testing to measure project progress
  - Motto: Testing must be planned/scheduled/managed
  - ⇒ A test suite is an artifact of software engineering, likewise code/docs/data/libraries/tutorials
- Quality School: testing as a quality process
  - Motto: A quality process for a quality product
  - ⇒ A test suite is a means to enact/control quality (checking internal/external attributes)
- Context-Driven School: testing as a stakeholder-oriented, adaptable process
  - Key question: which testing would be more valuable right now?
  - ⇒ A test suite is a contract with stakeholders, and follows a sequence of milestones
- Agile School: testing to facilitate change
  - Techniques: test automation, test-driven approaches
  - ⇒ A test suite is a necessary piece of code to achieve flexibility of design throughout process

Levels of testing, focussing on "what?"

Levels of testing

1. Unit testing: testing at the level of individual units (of functionality, or of code)
2. Integration testing: testing of the functionality provided by multiple integrated/interacting units
3. System testing: testing the system (whole or part) for correctness
   - Functional requirements: in a black-box way
   - Non-functional requirements: e.g., efficiency, reliability, usability, security
4. Stress testing: testing performance (memory, speed) in edge cases
5. Acceptance testing: testing features against expectations of users/stakeholders

Testing: concepts and definitions

ISTQB glossary: https://glossary.istqb.org

- **Test item/object**: a software item or artifact to be tested, also called SUT (system under test)
- **Test condition**: a testable aspect of a component (e.g., a feature)
- **Testability**: the degree to which (1) test conditions can be established and (2) tests can be performed to determine whether the test conditions have been met
- **Test case**: a set of preconditions, inputs, actions (where applicable), expected results and postconditions, defined based on test conditions
- **Test oracle (problem)**: determine expected result to be compared with that of the SUT
- **Test suite**: a set of test scripts/procedures to be executed in a specific test run; or a collection of test cases
- **Test run**: execution of a test suite on a version of the test object
- **Test tool**: hardware or software that supports 1+ test activities

→ You run a suite of test cases (using tools and oracles..) to verify a set of test conditions on a set of (testable) test items.

Outline

1 Testing basics

2 Test doubles and integration

3 Integration Testing

On dependency and the need of test doubles

On dependency and unit tests

- unit tests target single components (classes), working in isolation
- . . . otherwise we would test a collaboration, and failures would be about the collaboration
- necessarily, incrementally/modularity calls for testing:
  (i) components separately, and then, *(ii)* their collaboration

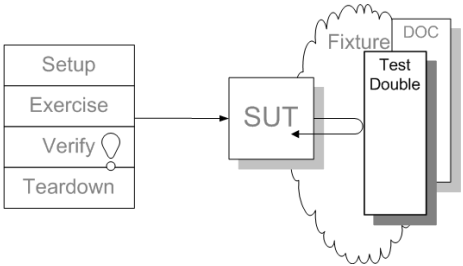
The basic idea of test doubles

- how do we unit-test components that need others for their correct functioning?
- we have to rely on a "test double", namely an alternative implementation for the component we depend upon, quickly setup just for the sake of tests and with predictable behaviour
- test doubles must be free of errors "by construction", otherwise we fall back to integration testing issues
- or, if testing is really needed for them, it should be much, much simpler than testing the actual version

Test Double

A reference model for test doubles

- SUT: System Under Test
- DOC: Depended-on Component
- TD: Test Double, an alternative version of the DOC
- The testing framework interacts with the SUT (which calls the DOC/Test double), and hence verifies what happened



Test Double – Why?

Reasons to rely on test doubles

- Sometimes it is just plain hard to test the SUT because it depends on other components that cannot be used in the test environment
  - they aren't available
  - they will not return the results needed for the test
  - executing them would have undesirable side effects
  - they would be too slow, or costly
  - they would break automation
  - ...
- In other cases, our test strategy requires us to have more control or visibility of the internal behaviour of the SUT
  - is the SUT calling the DOC in the proper way?
  - is the SUT correctly dealing with an error reported by the DOC?
  - what would happen if the DOC replies with an exception?

Test Double – How it works?

Metaphor

It is like the introduction of the Stunt Double in the movie industry: stunt doubles may not be able to act, but they know how to fall from great heights, crash a car, ...

Details

- For testing purposes, we can replace the real DOC (not the SUT!) with our equivalent: the TD
- During the fixture setup phase we replace the real DOC with our TD
  - Depending on the kind of test, we may hard-code the behaviour of the TD...
  - ...or we may configure it during the setup phase
- When the SUT interacts with the TD, it won't be aware that it isn't talking to the real DOC

Applicability is pervasive

Examples of components to "replace with a TD"

- GUI and CUI
- Random generation
- Complex algorithms
- External resources
  - databases
  - files
  - network
  - web resources
- Untested code
  - legacy/bad/early code
  - interfaces with no implementation yet
- in fact... any dependency

Types of Test Double

Stubs: emulate *replies* the DOC generates in response to certain calls

- Dummy: replaces DOC with an untested reference ("empty object")
- Stub: replaces DOC with a predefined responder (method-per-method)
- Fake object: replaces DOC with a light-weight version of it (sequence of results)

Mocks: emulate and examine *outgoing interactions* from the SUT to the DOC

- Spy: a proxy for a DOC, used to track the calls it receives
- Mock: replaces DOC with an object used to check if it is overall correctly used by the SUT

Creating TDs

- typically time consuming, often hampering applicability
- good libraries to create TDs are key: e.g., Mockito

Mockito

Tool

- just a library to create stubs/mocks
- `https://github.com/mockito/mockito`
- SBT dependency: `"org.mockito" % "mockito-core" % "3.+" % Test`

Key basic constructs

- `mock(C.class)` to create a dummy implementation
- `when(mockObj.meth()).thenResult(res)` to instruct a stubbed reply
- `spy(obj)` to insert a wrapper to detect events
- `verify(spyObj).meth()` checks if `meth` has been called on `spyObj`

A basic example

```java
public interface Device {
    void on() throws IllegalStateException;
    void off();
    boolean isOn();
    void reset();
}
```

```java
public interface FailingChecker {
    boolean attemptOn();
    void reset();
    String checkerName();
}
```

```java
import java.util.Objects;

public class StandardDevice implements Device {
    private FailingChecker failingChecker;
    private boolean on = false;

    public StandardDevice(FailingChecker failingChecker) {
        this.failingChecker = Objects.requireNonNull(failingChecker);
    }
    //...
```

Some unit tests with Mockito: Dummy

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

import org.mockito.Mockito;
import static org.mockito.Mockito.*;

public class StandardDeviceTest {

    private Device device;

    @Test
    @DisplayName("Device must specify a strategy")
    void testNonNullStrategy() {
        assertThrows(NullPointerException.class, () -> new StandardDevice(null));
    }

    @Nested
    class ShowcaseDummies {
        @Test
        @DisplayName("Device is initially off")
        void testInitiallyOff() {
            // an unused reference, a sort of "empty implementation"
            FailingChecker dummyFailingChecker = mock(FailingChecker.class);
            device = new StandardDevice(dummyFailingChecker);
            // checking that a device is on will not affect the strategy
            assertFalse(device.isOn());
        }
    }
```

Some unit tests with Mockito: Stub

```java
@Nested
class ShowcaseStubs {
    private FailingChecker stubFailingChecker;

    @BeforeEach
    void init(){
        this.stubFailingChecker = mock(FailingChecker.class);
        device = new StandardDevice(this.stubFailingChecker);
    }

    @Test
    @DisplayName("Device can be switched on")
    void testCanBeSwitchedOn() {
        // stubbing the test double, indicating "default behaviour"
        when(this.stubFailingChecker.attemptOn()).thenReturn(true);
        device.on();
        assertTrue(device.isOn());
    }

    @Test
    @DisplayName("Device won't switch on if failing")
    void testWontSwitchOn() {
        // multiple stubbing
        when(this.stubFailingChecker.attemptOn()).thenReturn(false);
        when(this.stubFailingChecker.checkerName()).thenReturn("mock");
        assertThrows(IllegalStateException.class, () -> device.on());
        assertEquals("StandardDevice{policy=mock, on=false}", device.toString());
    }
```

Some unit tests with Mockito: Fake

```java
    @Nested
    class ShowcaseFakes {
        private FailingChecker fakeFailingChecker;

        @BeforeEach
        void init(){
            this.fakeFailingChecker = mock(FailingChecker.class);
            device = new StandardDevice(this.fakeFailingChecker);
            // faking is more than stubbing: this object pretends to be the real one
            when(this.fakeFailingChecker.attemptOn()).thenReturn(true, true, false);
            when(this.fakeFailingChecker.checkerName()).thenReturn("mock");
        }

        @Test
        @DisplayName("Device switch on and off until failing")
        void testSwitchesOnAndOff() {
            IntStream.range(0, 2).forEach(i -> {
                device.on();
                assertTrue(device.isOn());
                device.off();
                assertFalse(device.isOn());
            });
            assertThrows(IllegalStateException.class, () -> device.on());
        }
    }
```

Some unit tests with Mockito: Spy

```java
@Nested
class ShowcaseSpies {
    private FailingChecker spyFailingChecker;

    @BeforeEach
    void init(){
        // the spy is essentially a proxy to the DOC, used to capture events
        this.spyFailingChecker = spy(new RandomChecker());
        device = new StandardDevice(this.spyFailingChecker);
    }

    @Test
    @DisplayName("AttemptOn is called as expected")
    void testReset() {
        device.isOn();
        // no interactions with the spy yet
        verifyNoInteractions(this.spyFailingChecker);
        try{
            device.on();
        } catch (IllegalStateException e){}
        // has attemptOn been called?
        verify(this.spyFailingChecker).attemptOn();
        device.reset();
        // have at least two method invocations be made?
        assertEquals(2,
            Mockito.mockingDetails(this.spyFailingChecker).getInvocations().size());
        //  Mockito.mockingDetails gives very powerful mechanisms...
    }
```

Some unit tests with Mockito: Mock

```java
@Nested
class ShowcaseMocks {
    private FailingChecker mockFailingChecker;

    @BeforeEach
    void init(){
        this.mockFailingChecker = spy(FailingChecker.class);
        device = new StandardDevice(this.mockFailingChecker);
        // the mock is a TD used to check you are collaborating as expected
        when(mockFailingChecker.attemptOn()).thenReturn(true, true, false);
        when(mockFailingChecker.checkerName()).thenReturn("mock");
    }

    @Test
    @DisplayName("attemptOn is called as expected")
    void testAttemptOn() {
        verify(this.mockFailingChecker, times(0)).attemptOn();
        device.on();
        verify(this.mockFailingChecker, times(1)).attemptOn();
        assertTrue(device.isOn());

        device.off();
        verify(this.mockFailingChecker, times(1)).attemptOn();
        device.on();
        verify(this.mockFailingChecker, times(2)).attemptOn();
        assertTrue(device.isOn());

        device.off();
        verify(this.mockFailingChecker, times(2)).attemptOn();
        assertThrows(IllegalStateException.class, () -> device.on());
        verify(this.mockFailingChecker, times(3)).attemptOn();
    }
}
```

Mockito annotations

```
    @Mock
    FailingChecker stubFailingChecker;
    @Spy
    RandomChecker spyRandomPolicy;

    @BeforeEach
    void init() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    void testMock() {
        device = new StandardDevice(this.stubFailingChecker);
        // multiple stubbing
        when(this.stubFailingChecker.attemptOn()).thenReturn(false);
        when(this.stubFailingChecker.checkerName()).thenReturn("mock");
        assertThrows(IllegalStateException.class, () -> device.on());
        assertEquals("StandardDevice{policy=mock, on=false}", device.toString());
    }

    @Test
    void testSpy() {
        device = new StandardDevice(this.spyRandomPolicy);
        // no interactions with the spy yet
        verifyNoInteractions(this.spyRandomPolicy);
        try{
            device.on();
        } catch (IllegalStateException e){}
        // has attemptOn been called?
        verify(this.spyRandomPolicy).attemptOn();
    }
}
```

Unit testing a device

```java
public class DeviceUnitTest {

    private Device device;
    @Mock FailingChecker checker;

    @BeforeEach
    void init(){
        MockitoAnnotations.openMocks(this);
        this.device = new StandardDevice(checker);
    }

    @Test
    void strategyShouldNotBeNull() {
        assertThrows(NullPointerException.class, () -> new StandardDevice(null));
    }

    @Test
    void initiallyOff() {
        assertFalse(device.isOn());
        verifyNoInteractions(checker);
    }

    @Test
    void canBeSwitchedOn() {
        when(checker.attemptOn()).thenReturn(true);
        device.on();
        verify(checker).attemptOn();
        assertTrue(device.isOn());
    }

    @Test
    void switchOnMayFail() {
        when(checker.attemptOn()).thenReturn(false);
        assertThrows(IllegalStateException.class, () -> device.on());
        verify(checker).attemptOn();
    }
}
```

Outline

1 Testing basics

2 Test doubles and integration

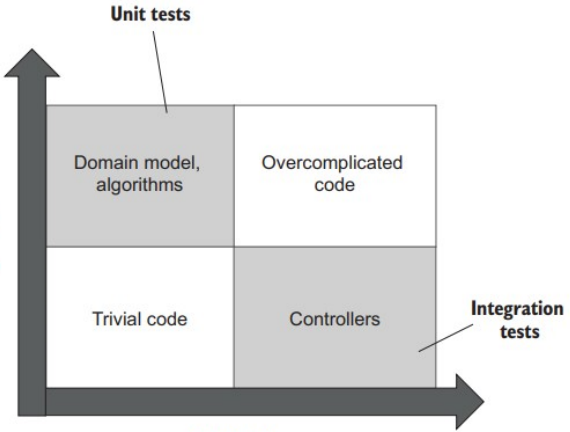
3 Integration Testing

Integration Testing

Unit testing is not enough

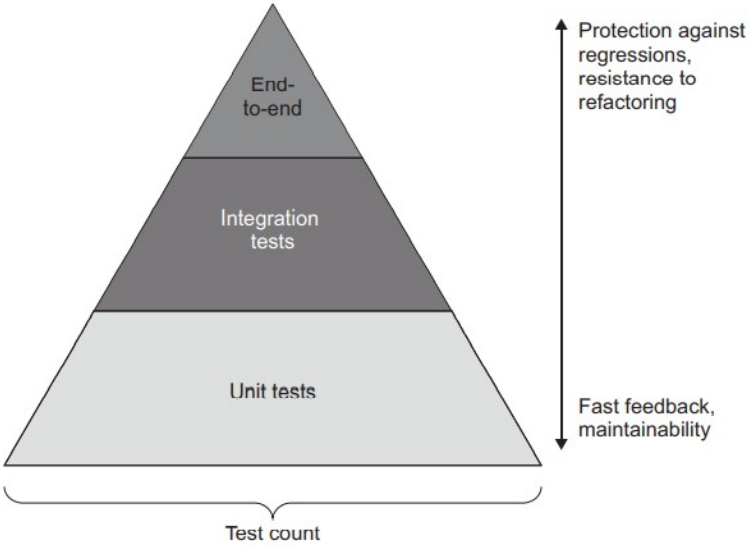
- We can never be sure a system works as a whole if we rely on unit tests exclusively
  - Unit tests are great at verifying business logic, and are great to guide quality development process, but they are not enough to test higher-level behaviour
  - We have to validate how different parts of a system integrate with each other and with external systems
  - Integration test has to be carried on in an automated way even if this is seemingly unfeasible due to unavailability of external resources
  - Integration test are not system tests, we just have to isolate collaborations
- Integration tests play an important role in the test suite
  - It is crucial to balance the number of unit and integration tests

Integration Testing vs. Unit Testing (1/3)



- A unit test is a test that meets the following three requirements:
  - verifies a single unit of behaviour,
  - does it quickly,
  - does it in isolation from other units and tests.
- If it doesn't meet all these three requirements, it falls into the category of integration tests

Integration Testing vs. Unit Testing (2/3)



Integration Testing vs. Unit Testing (3/3)

Unit tests vs integration tests

- integration tests are not to be confused with system tests
- an integration test verifies few collaborations
- that is, it carefully isolates the collaborations strictly needed to verify that a multi-component behaviour (or feature) is carried on correctly
- to do so, mocking is often used, to limit the scope of integration and to spy/check interactions with resources we cannot really exploit during testing

Integration tests vs system tests

- integration tests can be done at different levels, depending on what we need to verify
- they can test small collaborations up to bigger ones (e.g., the overall behaviour of a complex model)
- when there's no more isolation/mocking, and we are actually testing a whole system part, we actually entered system tests – though the distinction might not be so clean

Integration Testing

Description

- A manager handles devices, with commands to switch them all on or off
- On start and on errors, it files reports (e.g. for logging purposes)
- We want to test the correct collaboration between manager and devices, mocking checking and reporting

```java
public interface DeviceManager {
    void start();
    void addDevice(String name, Device device);
    void switchOnAll();
    void switchOnOne(String name);
    boolean isSwitchedOn(String name);
}
```

```java
public interface EventReporter {
    void reportStarted(String deviceName);
    void reportFailed(String deviceName, String
     reason);
}
```

Integration Testing – Example

```java
public class IntegrationTest {

    private DeviceManager deviceManager;
    private Device device;
    @Mock FailingChecker checker;
    @Mock EventReporter reporter;

    @BeforeEach
    void init(){
        MockitoAnnotations.openMocks(this);
        this.deviceManager = Factory.createDeviceManager(reporter);
        this.device = spy(new StandardDevice(checker));
        this.deviceManager.addDevice("dev-0", this.device);
    }

    @Test
    void managerCorrectlyReportsADeviceFailure() {
        when(this.checker.attemptOn()).thenReturn(false);
        this.deviceManager.start();
        this.deviceManager.switchOnAll();
        assertFalse(this.deviceManager.isSwitchedOn("dev-0"));

        verify(this.device).on();
        verify(this.checker).attemptOn();
        verify(this.reporter).reportStarted("dev-0");
        verify(this.reporter).reportFailed(eq("dev-0"), anyString());
    }
}
```

Integration testing best practices

- There are some general guidelines that can help to get the most out of integration tests:
  - Making domain model boundaries explicit
  - Reducing the number of layers
  - Eliminating circular dependencies
  - Using multiple act sections in a test
- As usual, best practices that are beneficial for tests also tend to improve the health of the code base in general
- Specifically, writing integration tests earlier supports so-called “design for testability”